

Samenvatting Theorie Web Apps - React

Bronnen: slides hoofdstuk 1 t.e.m. 7, [Master React in 5 days.pdf](#) en [Info examen\(1\) \(1\).pdf](#).

Examenfocus

Het examen bestaat uit twee delen:

- Deel 1: schriftelijk, gesloten boek. Verwacht kleine theorievragen, inzichtsvragen, meerkeuzevragen, code aanvullen, fouten zoeken/corrigeren en uitvoer voorspellen.
- Deel 2: programmeeroefening op laptop, open boek maar zonder internet. Offline cursusmateriaal, slides, codevoorbeelden, eigen notities en oefeningen mogen gebruikt worden.

Belangrijk: ken elk gezien React-begrip uit boek, slides en codevoorbeelden. Hoofdstuk 7-9 wordt niet gevraagd in het programmeerdeel, maar kan wel theoretisch gevraagd worden.

1. React Components

Wat is React?

React is een open source JavaScript-library van Meta voor het bouwen van interactieve user interfaces, vooral single-page applications.

Belangrijke kenmerken:

- Component-based: UI wordt opgebouwd uit herbruikbare componenten.
- Virtual DOM: React houdt een voorstelling van de UI in het geheugen bij.
- Reconciliation: React vergelijkt oude en nieuwe Virtual DOM en past daarna enkel de nodige wijzigingen toe in de echte DOM.
- Ecosysteem: onder andere React Router, Redux, Next.js, Gatsby, Material UI.
- Multi-platform: React voor web, React Native voor native mobiele apps.

Waarom componenten?

Een applicatie opdelen in componenten zorgt voor:

- beter begrijpbare code;
- onderhoudbaarheid;
- herbruikbaarheid;
- eenvoudiger testen en debuggen.

Een component is meestal een JavaScript-functie die JSX teruggeeft.

```
function Welcome() {  
  return <h1>Hello React</h1>;  
}  
  
export default Welcome;
```

Basisstructuur React-app

Typische bestanden:

- `index.html`: bevat meestal het root-element, bijvoorbeeld `<div id="root"></div>`.
- `index.js` of `main.jsx`: maakt de React root en rendert de app.
- `App.js` of `App.jsx`: hoofdcomponent van de applicatie.

Voorbeeld:

```
import React from "react";  
import ReactDOM from "react-dom/client";  
import App from "./App";  
  
const root = ReactDOM.createRoot(document.getElementById("root"));  
root.render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```

`React.StrictMode` helpt tijdens ontwikkeling om fouten en onzuivere code sneller zichtbaar te maken.

Project starten

Legacy manier uit de cursus:

```
npm install -g create-react-app  
create-react-app naam  
cd naam  
npm start
```

Nieuwe manier met Vite:

```
npm create vite@latest naam -- --template react  
cd naam  
npm run dev
```

JSX

JSX staat voor JavaScript and XML. Het laat toe om HTML-achtige code in JavaScript te schrijven.

```
const name = "Joshua";

function App() {
  return <h1>Hallo {name}</h1>;
}
```

Regels:

- JavaScript-expressies plaats je tussen {}.
- Een component moet één root-element teruggeven.
- Gebruik `className` in plaats van `class`.
- Inline CSS schrijf je als object.
- Componentnamen starten met een hoofdletter.

Props

Props zijn attributen die je doorgeeft aan een component.

```
function Counter({ init }) {
  return <p>Startwaarde: {init}</p>;
}

function App() {
  return (
    <>
      <Counter init="5" />
      <Counter init={5} />
    </>
  );
}
```

Verschil:

- `init="5"` geeft een string door.
- `init={5}` geeft een JavaScript-getal door.

State met useState

State is data waarvan een wijziging de component opnieuw kan renderen.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
}
```

```
function increment() {
  setCount(count + 1);
}

return <button onClick={increment}>{count}</button>;
}
```

Belangrijk:

- `count` mag je niet rechtstreeks aanpassen met `count++` of `count = count + 1`.
- Gebruik altijd de setter, hier `setCount`.
- Een state-update is niet onmiddellijk zichtbaar in dezelfde uitvoeringsstap.
- Bij opeenvolgende updates gebruik je best de functionele vorm.

```
setCount((count) => count + 1);
setCount((count) => count + 1);
```

Dit telt effectief twee keer op. Met `setCount(count + 1)` twee keer na elkaar gebruik je mogelijk twee keer dezelfde oude waarde.

Side effects en useEffect

Een renderfunctie moet zo zuiver mogelijk zijn: dezelfde input geeft dezelfde output. Timers, netwerkverzoeken, DOM-focus en subscriptions zijn side effects en horen in `useEffect`.

Timer met cleanup:

```
import { useEffect, useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  useEffect(function () {
    const timer = setInterval(function () {
      setCount((count) => count + 1);
    }, 1000);

    return function () {
      clearInterval(timer);
    };
  }, []);

  return <p>{count}</p>;
}
```

Let op:

- Zonder cleanup kunnen meerdere timers tegelijk blijven lopen.
- Zonder dependency array draait `useEffect` na elke render.
- Met `[]` draait de effect bij de eerste render.
- Met `[value]` draait de effect opnieuw wanneer `value` wijzigt.

2. JSX Code

Fragmenten

Een component moet één root teruggeven. Je kan een extra `<div>` gebruiken, maar dat voegt een echt element toe aan de DOM. Fragmenten lossen dit op.

Mogelijkheden:

```
<React.Fragment>...</React.Fragment>
<Fragment>...</Fragment>
<>...</>
```

Verschillen:

- `<React.Fragment>` vereist `import React from "react"`.
- `<Fragment>` vereist `import { Fragment } from "react"`.
- `<>...</>` is kort, maar kan geen attributen zoals `key` krijgen.

Conditie in JSX

Ternary operator:

```
function Message({ loggedIn }) {
  return <p>{loggedIn ? "Welkom" : "Gelieve in te loggen"}</p>;
}
```

Conditioneel tonen met `&&`:

```
{error && <p className="error">{error}</p>}
```

Zelfoproepende functie kan ook, maar is vaak minder leesbaar:

```
{{() => {
  if (score >= 10) return <p>Geslaagd</p>;
  return <p>Niet geslaagd</p>;
}}()}}
```

Lussen in JSX

Gebruik meestal `map`.

```
const names = ["Ana", "Bo", "Cem"];

function App() {
  return (
    <ul>
      {names.map((name) => (
        <li key={name}>{name}</li>
      ))}
    </ul>
  );
}
```

Belangrijk:

- Elk element in een lijst heeft een unieke en stabiele `key` nodig.
- Gebruik de index alleen als key wanneer de lijst nooit wijzigt, verwijderd of hersorteerd wordt.
- Voor reeksen kan je werken met `[...Array(10).keys()]`.

```
{[...Array(10).keys()].map((i) => (
  <span key={i}>{i + 1}</span>
))}
```

Styling in JSX

Inline styling:

```
<p style={{ color: "red", fontSize: "20px" }}>Tekst</p>
```

Let op:

- Buitenste `{}`: JavaScript in JSX.
- Binnenste `{}`: JavaScript-object.
- CSS-eigenschappen worden camelCase: `font-size` wordt `fontSize`.

Externe CSS:

```
import "./Counter.css";

function Counter() {
  return <p className="counter">0</p>;
}
```

3. Event Handling

Events verwerken

React-events schrijf je in camelCase:

- HTML: `onclick`
- React: `onClick`

Voorbeeld:

```
function ButtonCounter() {  
  const [count, setCount] = useState(0);  
  
  function handleClick() {  
    setCount((count) => count + 1);  
  }  
  
  return <button onClick={handleClick}>{count}</button>;  
}
```

Belangrijk:

- Schrijf `onClick={handleClick}`.
- Schrijf niet `onClick={handleClick()}`, want dan voer je de functie meteen uit tijdens render.

Input verwerken

Controlled component:

```
function NameInput() {  
  const [name, setName] = useState("");  
  
  function handleChange(event) {  
    setName(event.target.value);  
  }  
  
  return <input value={name} onChange={handleChange} />;  
}
```

Belangrijk:

- `event.target.value` bevat de huidige waarde van het inputveld.
- De React-state is de bron van waarheid.
- Met `onChange` hou je de state synchroon met de input.

Veelgebruikte events

- `onClick`: klikken op knop of element.

- `onChange`: invoerwaarde wijzigt.
- `onKeyDown`: toets wordt ingedrukt.
- `onBlur`: element verliest focus.
- `onFocus`: element krijgt focus.

4. React Hooks

Wat is een hook?

Een hook is een React-functie waarmee function components extra mogelijkheden krijgen, zoals state, side effects, context, refs of complexere state-logica.

Hooks beginnen met `use`, bijvoorbeeld:

- `useState`
- `useEffect`
- `useContext`
- `useRef`
- `useReducer`

Hoofdregel van hooks

Hooks moeten altijd in dezelfde volgorde uitgevoerd worden.

Daarom:

- gebruik hooks bovenaan in een React-component;
- gebruik hooks niet in `if`, `for`, `while`, gewone nested functies of callbacks;
- gebruik hooks alleen in React-componenten of custom hooks.

Fout:

```
if (loggedIn) {  
  const [name, setName] = useState("");  
}
```

Goed:

```
const [name, setName] = useState("");  
  
if (!loggedIn) {  
  return <p>Niet ingelogd</p>;  
}
```

`useState`

Gebruik voor reactieve variabelen.


```
const [value, setValue] = useState(0);
```

Elke instantie van een component heeft zijn eigen state.

useEffect

Gebruik voor side effects:

- timer starten/stoppen;
- data ophalen;
- manuele DOM-acties;
- subscriptions;
- logging na render.

Vormen:

```
useEffect(() => {  
  // na elke render  
});  
  
useEffect(() => {  
  // enkel na eerste render  
}, []);  
  
useEffect(() => {  
  // na eerste render en telkens wanneer id wijzigt  
}, [id]);
```

Cleanup:

```
useEffect(() => {  
  const timer = setInterval(doSomething, 1000);  
  return () => clearInterval(timer);  
}, []);
```

useRef

Gebruik voor een waarde die je wil bewaren zonder render te triggeren, of om een DOM-element te benaderen.

Focus op input:

```
import { useEffect, useRef } from "react";  
  
function AutoFocusInput() {
```

```
const inputRef = useRef();

useEffect(() => {
  inputRef.current.focus();
}, []);

return <input ref={inputRef} />;
}
```

Verschil met state:

- `useState`: wijziging triggert render.
- `useRef`: wijziging triggert geen render.

useContext

Gebruik om data te delen met componenten die dieper in de componentboom zitten, zonder props door elke tussenlaag te moeten doorgeven.

Typisch patroon:

```
import { createContext, useContext } from "react";

const ThemeContext = createContext("light");

function Child() {
  const theme = useContext(ThemeContext);
  return <p>Theme: {theme}</p>;
}

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Child />
    </ThemeContext.Provider>
  );
}
```

useReducer

Gebruik bij complexere state, vooral wanneer updates afhangen van acties.

```
function reducer(state, action) {
  if (action.type === "increment") {
    return { count: state.count + 1 };
  }

  if (action.type === "decrement") {
    return { count: state.count - 1 };
  }
}
```

```
}

return state;
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });

  return (
    <>
      <button onClick={() => dispatch({ type: "decrement" })}>-</button>
      <span>{state.count}</span>
      <button onClick={() => dispatch({ type: "increment" })}>+</button>
    </>
  );
}
```

Custom hook

Een custom hook is een eigen functie die hooks gebruikt en zelf ook met `use` begint.

```
function useCounter(init = 0) {
  const [count, setCount] = useState(init);

  function increment() {
    setCount((count) => count + 1);
  }

  return { count, increment };
}
```

5. Practical Application: Task List

Componentstructuur

De task-list-app uit de cursus gebruikt typisch:

- `App`: bewaart de lijst in state en bevat de knop `Add Item`.
- `Items`: toont de volledige lijst.
- `Item`: toont één item met `Modify` en `Remove`.

Items bewaren

```
const [items, setItems] = useState([]);
```

Item toevoegen:

```
function add() {  
  const item = "Item " + (items.length + 1);  
  setItems([...items, item]);  
}
```

Belangrijk: maak een nieuwe array. Alleen `items.push(item)` is niet genoeg, want React ziet dan mogelijk dezelfde array-referentie.

Lijst tonen

```
function Items({ items }) {  
  return (  
    <ul>  
      {items.map((item) => (  
        <Item key={item.id} item={item} />  
      ))}  
    </ul>  
  );  
}
```

Item verwijderen

Gebruik `filter`.

```
function removeItem(id) {  
  setItems((items) => items.filter((item) => item.id !== id));  
}
```

Item wijzigen

Gebruik `map`.

```
function updateItem(id, newName) {  
  setItems((items) =>  
    items.map((item) =>  
      item.id === id ? { ...item, name: newName } : item  
    )  
  );  
}
```

Unieke keys

Gebruik geen array-index als `key` wanneer items verwijderd of gewijzigd kunnen worden.

Slecht bij dynamische lijsten:

```
items.map((item, index) => <Item key={index} />)
```

Beter:

```
items.map((item) => <Item key={item.id} item={item} />)
```

Waarom? React gebruikt **key** om items te herkennen. Als keys veranderen na verwijderen, kan React de verkeerde component hergebruiken en krijg je foutieve weergave.

Modify met input

Typisch patroon:

```
function Item({ item, onUpdate, onRemove }) {
  const [editing, setEditing] = useState(false);
  const [text, setText] = useState(item.name);

  function save() {
    setEditing(false);
    onUpdate(item.id, text);
  }

  return (
    <li>
      {editing ? (
        <input
          value={text}
          onChange={(event) => setText(event.target.value)}
          onBlur={save}
        />
      ) : (
        <span>{item.name}</span>
      )}
      <button onClick={() => setEditing(true)}>Modify</button>
      <button onClick={() => onRemove(item.id)}>Remove</button>
    </li>
  );
}
```

Focus op input na Modify

Gebruik **useRef** en **useEffect**.

```
const inputRef = useRef();

useEffect(() => {
  if (editing) {
    inputRef.current.focus();
  }
}, [editing]);
```

```
<input ref={inputRef} value={text} onChange={handleChange} />
```

6. JavaScript Reminders

let, var en const

- **var**: function scope, oudere stijl, vermijden waar mogelijk.
- **let**: block scope, waarde kan veranderen.
- **const**: block scope, variabele kan niet opnieuw toegewezen worden.

```
let count = 0;
count = 1;

const name = "React";
```

Bij objecten en arrays betekent **const** niet dat de inhoud onveranderlijk is:

```
const items = [];
items.push("A"); // kan
// items = []; // kan niet
```

Object destructuring

```
const person = { name: "Ana", age: 20 };
const { name, age } = person;
```

Handig bij props:

```
function User({ name, age }) {
  return <p>{name} is {age}</p>;
}
```

Array destructuring

```
const values = [10, 20];  
const [first, second] = values;
```

React gebruikt dit bij `useState`:

```
const [count, setCount] = useState(0);
```

Spread operator

Array kopiëren/toevoegen:

```
const numbers = [1, 2, 3];  
const copy = [...numbers];  
const extra = [...numbers, 4];
```

Object kopiëren/wijzigen:

```
const user = { name: "Ana", age: 20 };  
const updated = { ...user, age: 21 };
```

In React is spread belangrijk om nieuwe referenties te maken, zodat React wijzigingen detecteert.

Import en export

Named export:

```
export function sum(a, b) {  
  return a + b;  
}
```

Import:

```
import { sum } from "./math";
```

Default export:

```
export default App;
```

Import:

```
import App from "./App";
```

Vershil:

- Named export moet met dezelfde naam tussen `{}` geïmporteerd worden.
- Default export mag bij import een eigen naam krijgen.

Arrow functions

```
const add = (a, b) => a + b;
```

Met body:

```
const add = (a, b) => {  
  return a + b;  
};
```

In React vaak gebruikt bij events:

```
<button onClick={() => removeItem(id)}>Remove</button>
```

map en filter

map: maakt een nieuwe array op basis van elk element.

```
const doubled = numbers.map((n) => n * 2);
```

filter: maakt een nieuwe array met enkel elementen die voldoen aan een voorwaarde.

```
const positives = numbers.filter((n) => n > 0);
```

In React:


```
{items.map((item) => (  
  <li key={item.id}>{item.name}</li>  
))}
```

Promise, async en await

Een Promise stelt een toekomstige waarde voor: pending, fulfilled of rejected.

```
fetch("/api/items")  
  .then((response) => response.json())  
  .then((data) => console.log(data))  
  .catch((error) => console.error(error));
```

Met `async` en `await`:

```
async function loadItems() {  
  try {  
    const response = await fetch("/api/items");  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error(error);  
  }  
}
```

In React combineer je data ophalen vaak met `useEffect`.

```
useEffect(() => {  
  async function loadItems() {  
    const response = await fetch("http://localhost:3001/items");  
    const data = await response.json();  
    setItems(data);  
  }  
  
  loadItems();  
}, []);
```

7. React Routes

Hoofdstuk 7 is volgens de exameninfo niet voor het programmeerdeel, maar de principes kunnen wel in het theorieeldeel gevraagd worden.

Wat is React Router?

React Router is een library voor routing in React-applicaties.

Mogelijkheden:

- meerdere pagina's/views in een single-page application;
- URL-parameters verwerken;
- query strings verwerken;
- browsergeschiedenis en navigatie beheren;
- nested routes en layouts;
- beveiligde routes voor authenticatie.

Installatie:

```
npm install react-router-dom
```

Import:

```
import { BrowserRouter, Routes, Route, Link, NavLink } from "react-router-dom";
```

Eenvoudige route

```
function App() {  
  return (  
    <BrowserRouter>  
      <nav>  
        <Link to="/">Home</Link>  
        <Link to="/about">About</Link>  
      </nav>  
  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/about" element={<About />} />  
      </Routes>  
    </BrowserRouter>  
  );  
}
```

Elementen:

- **BrowserRouter**: root-element dat routing mogelijk maakt.
- **Link**: navigatie zonder volledige pagina-refresh.
- **NavLink**: zoals **Link**, maar handig voor actieve styling.
- **Routes**: bevat de route-definities.
- **Route**: koppelt een pad aan een component.

NavLink

```
<NavLink className={({ isActive }) => isActive ? "active" : ""} to="/about">
  About
</NavLink>
```

Handig voor menu's, breadcrumbs en tabbladen.

Geneste routes

Gebruik child routes en `Outlet`.

```
import { Outlet, Link } from "react-router-dom";

function Products() {
  return (
    <>
      <Link to="cars">Cars</Link>
      <Link to="bikes">Bikes</Link>
      <Outlet />
    </>
  );
}
```

```
<Route path="/products" element={<Products />}>
  <Route path="cars" element={<CarProducts />} />
  <Route path="bikes" element={<BikeProducts />} />
</Route>
```

`Outlet` bepaalt waar de child route getoond wordt.

URL-parameters

Route:

```
<Route path="/customer/:name" element={<Customer />} />
```

Component:

```
import { useParams } from "react-router-dom";

function Customer() {
  const { name } = useParams();
  return <p>Klant: {name}</p>;
}
```

Voorbeeld-URL:

```
http://localhost:3000/customer/Tobias
```

Hier is **Tobias** de URL-parameter.

Data doorgeven

Mogelijkheden:

- via props;
- via URL-parameters;
- via query parameters;
- via **useContext**;
- via backend/API.

Frontend en backend

Typische opzet:

- React frontend draait op een eigen poort.
- Backend met bijvoorbeeld Node.js/Express draait op een andere poort.
- Frontend haalt data op met **fetch**.
- Vaak wordt JSON gebruikt als uitwisselingsformaat.

Voorbeeld:

```
useEffect(() => {
  fetch("http://localhost:3001/tasks")
    .then((response) => response.json())
    .then((data) => setTasks(data));
}, []);
```

Next.js is een React-framework dat bovenop React een projectstructuur en server-side mogelijkheden toevoegt.

8. PWA - Introduction

Hoofdstuk 8 is vooral theoriegericht voor het examen. Het kan gevraagd worden in deel 1, maar de basisconfiguratie kan ook nuttig zijn als je offline een bestaand project moet herkennen.

Wat is een PWA?

Een Progressive Web App is een webapplicatie met HTML, CSS en JavaScript die zich app-achtig gedraagt.

Kenmerken:

- installeerbaar op startscherm;
- gedeeltelijk of volledig offline bruikbaar;
- gebruikt browsertechnologie;
- kan service workers gebruiken voor caching en updates;
- kan push-notificaties ondersteunen, afhankelijk van platform/browser.

Een PWA is geen aparte programmeertaal of framework. Het is een combinatie van webtechnologie, best practices en browser-API's.

Vergelijking app-types

- Web app: lage ontwikkelkost, draait in browser, beperkte hardwaretoegang, meestal niet installeerbaar als echte app.
- PWA: webtechnologie + service worker + manifest, installeerbaar, offline cache mogelijk, hardwaretoegang blijft browserafhankelijk.
- Hybride app: webtechnologie in WebView met native bridge, publicatie in store mogelijk.
- React Native app: JavaScript/TypeScript + React-concepten, maar native UI-componenten; geen WebView voor gewone UI.
- Echte native app: Kotlin/Java voor Android of Swift/Xcode voor iOS, hoogste platformcontrole maar duurder en aparte codebases.

Technische vereisten

Een PWA heeft typisch:

- HTTPS, behalve bij testen op `localhost`;
- een geldig web app manifest;
- een service worker;
- iconen en assets voor verschillende devices;
- cachingstrategie voor offline gebruik.

Manifest bevat minimaal zaken zoals:

```
{
  "name": "my-react-pwa-app",
  "short_name": "my-react-pwa-app",
  "start_url": "/",
  "display": "standalone",
  "theme_color": "#ffffff",
  "icons": []
}
```

Service worker

Een service worker is een JavaScript-bestand dat in de achtergrond draait. Het kan onder andere:

- bestanden cachen;
- offline gedrag ondersteunen;
- pushmeldingen verwerken;
- updates van de app beheren.

Zonder service worker is een webapp meestal niet echt installable als PWA.

Workbox

Workbox is een library van Google die caching en service workers vereenvoudigt. Vite PWA gebruikt Workbox om automatisch service workers en cachingregels te genereren.

Vite PWA

Project aanmaken volgens de slides:

```
npm create @vite-pwa/pwa@latest naam -- --template react
cd naam
npm install
npm run dev
```

Productie testen:

```
npm run build
npm run preview
```

Belangrijk:

- `npm run dev`: gewone ontwikkelserver.
- `npm run build`: maakt productie-build in `dist`.
- `npm run preview`: simuleert productie; beter om service worker/cache te testen.

Voorbeeld uit de oefeningen:

```
import { VitePWA } from "vite-plugin-pwa";
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [
    react(),
    VitePWA({
      registerType: "prompt",
      manifest: {
        name: "my-react-pwa-app",
        short_name: "my-react-pwa-app",
      },
    }),
  ],
});
```

```

    start_url: "/",
    display: "fullscreen",
    theme_color: "#ffffff",
  },
  workbox: {
    globPatterns: ["**/*.{js,css,html,svg,png,ico}"],
    cleanupOutdatedCaches: true,
    clientsClaim: true,
  },
}),
],
});

```

generateSW vs injectManifest

- **generateSW**: Workbox genereert automatisch een service worker met standaard cachingregels.
- **injectManifest**: je schrijft zelf service-workerlogica; Workbox injecteert enkel de lijst met te precachen bestanden.

Gebruik **generateSW** voor eenvoudige standaard-PWA's. Gebruik **injectManifest** wanneer je zelf controle nodig hebt over caching, background sync of push.

Updates

- Prompt for update: gebruiker krijgt melding wanneer nieuwe versie beschikbaar is.
- Auto update: service worker update op de achtergrond.
- Periodic updates: browser controleert periodiek of er nieuwe service worker/appversie is.

Gegenereerde bestanden na build

In **dist** kunnen onder andere verschijnen:

- **manifest.webmanifest**;
- **registerSW.js**;
- **sw.js**;
- **workbox.js**;
- aangepaste **index.html** met manifest- en service-workerverwijzingen.

Belangrijke PWA-valkuilen

- PWA-features vragen vaak HTTPS.
- Offline gedrag test je best via productie-build en preview.
- Een PWA die in Chrome is geïnstalleerd is een andere instantie dan dezelfde PWA geïnstalleerd vanuit een andere browser.
- Publicatie in een store kan, maar vraagt soms een wrapper zoals Capacitor, Cordova of PWABuilder.

9. React Native - Introduction

Hoofdstuk 9 is vooral theoriegericht voor deel 1. Voor deel 2 wordt React Native volgens de exameninfo niet gevraagd, maar de principes kunnen wel bevraagd worden.

Wat is React Native?

React Native is een framework voor mobiele apps met JavaScript/TypeScript en React-concepten. Het werd ontwikkeld door Meta en gebruikt React om native UI-componenten aan te sturen.

Ondersteunde platformen:

- Android;
- iOS;
- macOS;
- Windows;
- Web.

Voordelen

- Een codebase voor meerdere platformen.
- Snelle ontwikkeling met hot reloading.
- Native performance voor gewone UI.
- Grote community.
- React-hooks en businesslogica zijn vaak herbruikbaar.

Beperkingen

- Niet elke native feature is standaard beschikbaar.
- Soms moet je native modules of platformcode gebruiken.
- Minder geschikt voor zeer grafisch intensieve apps.
- UI moet herschreven worden tegenover React web.

React web vs React Native

React web	React Native
HTML-elementen zoals <code>div</code> , <code>p</code> , <code>button</code>	Native componenten zoals <code>View</code> , <code>Text</code> , <code>Button</code> , <code>Pressable</code>
CSS-bestanden of inline CSS	<code>StyleSheet.create()</code> met JavaScript-objecten
DOM	Native UI via Fabric
Browser runtime	Mobiele runtime zoals Hermes, JSC of V8
<code>onClick</code>	<code>onPress</code>
React Router	Vaak React Navigation

Expo

In de cursus wordt Expo gekozen als ontwikkelplatform.

Expo biedt:

- eenvoudige projectstart;
- Expo Go om apps te testen;
- development builds;

- EAS voor cloud-builds;
- standaardmodules en tooling.

Project aanmaken:

```
npx create-expo-app@latest my-react-native-app --template blank
cd my-react-native-app
npm run android
```

Andere commando's:

```
npm run ios
npm run web
```

Voor iOS-builds heb je macOS nodig, tenzij je via Expo Go of EAS werkt.

Templates

- **default**: standaard Expo-template, vaak met TypeScript en routing.
- **blank**: minimale start.
- **blank-typescript**: minimale start met TypeScript.
- **tabs**: tabnavigatie met Expo Router.
- **bare-minimum**: geeft native **android** en **ios** mappen.

Migreren van React naar React Native

Herbruikbaar:

- state management met hooks;
- API-calls met **fetch**;
- businesslogica;
- helpers;
- validatiefuncties.

Herschrijven:

- HTML naar native componenten;
- CSS naar **StyleSheet**;
- DOM-manipulatie;
- web-specifieke events;
- routing naar React Navigation of Expo Router.

Basisvoorbeeld

```
import { useEffect, useState } from "react";
import { Text } from "react-native";
```

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const timer = setInterval(() => {
      setCount((count) => count + 1);
    }, 1000);

    return () => clearInterval(timer);
  }, []);

  return <Text>The counter is set to: {count}</Text>;
}
```

App met root **View**:

```
import { View, Button, Alert } from "react-native";

function App() {
  return (
    <View>
      <Button title="Click me" onPress={() => Alert.alert("Hello")} />
    </View>
  );
}
```

Belangrijk:

- JSX bevat geen HTML.
- Een scherm start vaak met **View**, **SafeAreaView** of **ScrollView**.
- Tekst moet in een **Text** component staan.
- Styling gebeurt via objecten, niet via gewone CSS-bestanden.

Typische Examenvragen En Valkuilen

Wat is fout?

Fout:

```
<button onClick={handleClick()}>Klik</button>
```

Waarom fout? De functie wordt meteen uitgevoerd tijdens render.

Correct:

```
<button onClick={handleClick}>Klik</button>
```

State rechtstreeks aanpassen

Fout:

```
count++;  
items.push(newItem);
```

Correct:

```
setCount((count) => count + 1);  
setItems((items) => [...items, newItem]);
```

Geen key in lijst

Fout:

```
items.map((item) => <li>{item.name}</li>)
```

Correct:

```
items.map((item) => <li key={item.id}>{item.name}</li>)
```

Index als key bij wijzigende lijst

Risico:

```
items.map((item, index) => <Item key={index} item={item} />)
```

Kan fout lopen wanneer items verwijderd, toegevoegd of hersorteerd worden. Gebruik een stabiele unieke id.

Hook in conditie

Fout:

```
if (show) {  
  useEffect(() => {}, []);  
}
```

```
}
```

Correct:

```
useEffect(() => {  
  if (show) {  
    // doe iets  
  }  
}, [show]);
```

Input zonder onChange

Als je **value** zet op een input, moet je meestal ook **onChange** voorzien.

```
<input value={text} onChange={(event) => setText(event.target.value)} />
```

useEffect zonder dependency array

```
useEffect(() => {  
  setCount(count + 1);  
});
```

Dit kan een oneindige renderlus veroorzaken: render -> effect -> state update -> render -> effect -> ...

Props vs state

- Props komen van een parent component.
- State leeft in de component zelf.
- Props zijn read-only voor de child.
- State wijzig je met een setter.

Controlled vs uncontrolled input

Controlled:

```
<input value={text} onChange={(event) => setText(event.target.value)} />
```

React-state bepaalt de waarde.

Uncontrolled:

```
<input ref={inputRef} />
```

DOM bewaart de waarde; je leest ze eventueel via ref.

Snelle Checklist Voor Het Programmeerdeel

- Kan ik een React-project starten?
- Kan ik componenten maken en importeren/exporteren?
- Kan ik props doorgeven en destructuren?
- Kan ik `useState` gebruiken voor formulieren, tellers en lijsten?
- Kan ik events correct koppelen zonder de handler meteen uit te voeren?
- Kan ik lijsten tonen met `map` en correcte `key`?
- Kan ik arrays immutabel aanpassen met `spread`, `map` en `filter`?
- Kan ik `useEffect` gebruiken zonder oneindige lus?
- Kan ik `cleanup` schrijven voor timers?
- Kan ik met `useRef` focus geven aan een input?
- Kan ik React Router-code herkennen en uitleggen?